

Distributed Shared White Board

1408063 Soyeon Park

1. Introduction

In this project, we implemented a whiteboard that can be shared among multiple users using Java Remote Method Invocation (RMI). A shared whiteboard is a system that allows multiple users to draw on canvas and enter text at the same time.

RMI is a distributed object technology that allows us to call methods for remote objects over the network. This allowed us to understand and implement the main concepts of distributed systems, such as remote method calling, network communication, synchronization, and more.

RMI is an API that Java provides for building distributed systems, allowing remote JVMs to call methods for objects running on local JVMs. This allows developers to call remote objects as if they were calling local objects, simplifying the implementation of distributed applications. RMI consists of the following key components:

Remote Interface: Defines the method provided by the remote object.

Remote Object: An object that implements a remote interface.

RMI Registry: A registry that allows you to register and retrieve remote objects.

The main advantages of RMI are that object-oriented designs can be applied to distributed systems by abstracting network communication, and that they reduce the burden on developers by generating automated stubs. Moreover, the object-oriented properties of Java can be used as they are, providing a consistent programming model.

However, since RMI communicates using specific ports, communication may be blocked depending on firewall settings, and method calls over the network may suffer from poor performance compared to local calls. Furthermore, RMI is Java dependent, resulting in a lack of interoperability with other languages. Despite these shortcomings, RMI is a powerful tool for easily building distributed systems in Java environments.

Through this project, I was able to learn in-depth how to build distributed systems using RMI. While implementing the shared whiteboard system, various computer science concepts such as user management, concurrency control, GUI design, etc. could be integrated and applied. Through this, I was able to gain practical experience in the design and implementation of distributed systems using RMI, and I was able to understand the strengths and limitations of RMI.

This project has been a great help in increasing our understanding of distributed system deployment using RMI, and we have been able to build our overall knowledge of distributed systems through the process of designing and implementing distributed applications that actually work.

2. Method

2.1 Components of the System

The system is a multi-user whiteboard application based on a client-server architecture, with its main components divided into managers and clients. System design focuses on allowing multiple users to access servers simultaneously and perform tasks such as drawing or entering text efficiently.

① Manager (Create White Board)

The manager is responsible for creating and managing shared whiteboards.

The manager's key functions and responsibilities include:

Create Whiteboard: When you first start the system, the manager creates a new whiteboard.

User Management: Managers can approve or reject new users when they make access requests; managers also have the right to expel certain users from the system.

File Management: Manager can save the contents of a whiteboard as a file, or retrieve an existing file. Manager can also create a new whiteboard or close the whiteboard that you are currently working on.

Manage synchronization: Manager synchronizes the current state of the whiteboard to all clients so that all users can see the same screen.

Draw and enter text: The manager can do everything the client can, for example, draw on a whiteboard or enter text. These changes are synchronized to all users in real time.

② Client (Join White Board)

Clients are responsible for users accessing and interacting with the whiteboard.

The key features and responsibilities of the client are:

Access Request: The user sends the access request to the manager through the client. Once approved by the manager, you can access the system as a user.

Draw and enter text: Users can draw or enter text on the whiteboard via the client interface. These changes are synchronized to all users in real time.

Chatting: Clients can communicate with other users by chatting. The entered chat will be sent to all users in real time.

Check User List: Clients can view the list of users who currently have access to the whiteboard, which is updated in real time.

Keep synchronization: Clients periodically receive the current status of the whiteboard from their manager and update their screens.

2.2. Class Design

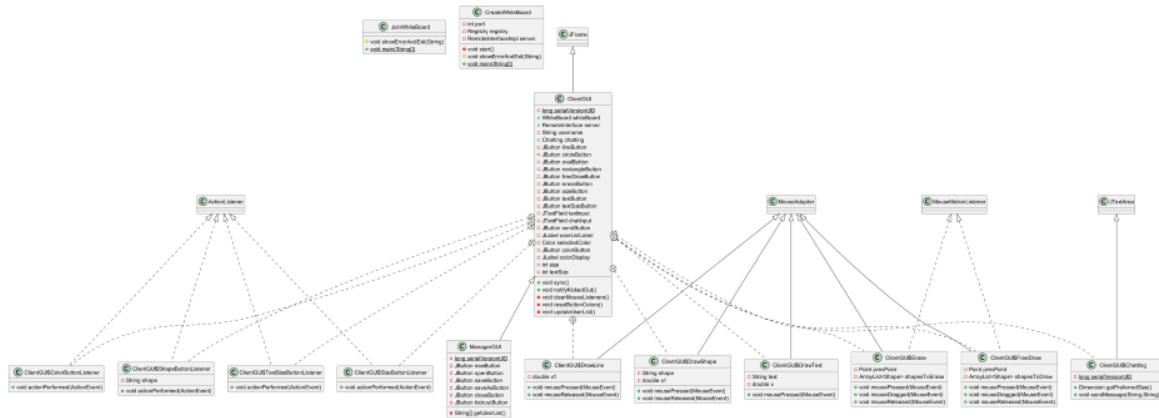


Figure 1. Class Diagram for Create White Board, Join White Board, and GUI

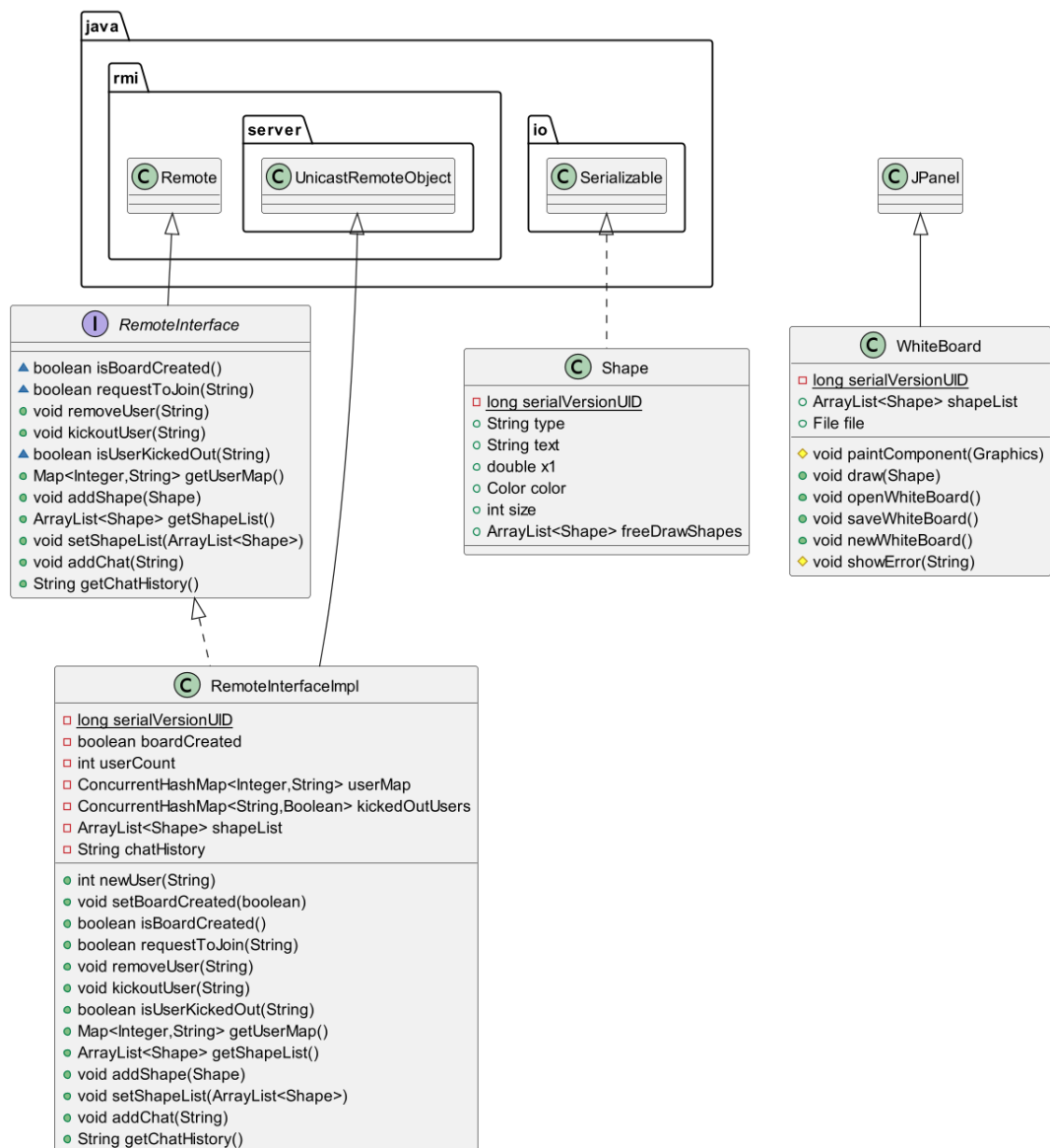


Figure 2. Class Diagram for Remote Interface, Whiteboard and Shape

CreateWhiteBoard	Create a manager client, register the server object in the RMI registry. It also initializes the manager GUI and performs synchronization functions.
JoinWhiteBoard	Create a user client and send a connection request to the manager. If the connection is approved, initialize the client GUI and perform the synchronization function.
ManagerGUI	Provides a manager interface and performs whiteboard file management and user management functions.
ClientGUI	Provides a user interface and performs functions such as drawing, text input, and chatting.
WhiteBoard	A class that implements a real whiteboard and provides the features to draw figures and text.
RemoteInterface	Interface that defines the remote method called through the RMI.
RemoteInterfaceImpl	Implement remote methods, manage the status of the whiteboard, and handle client requests.
Shape	A class that defines shapes and text objects drawn on the whiteboard.

Table 1. Class Design

These classes use Remote Method Invocation (RMI) to process client-to-server communications, allowing clients to call the remote server's methods to change the status of the whiteboard or request information. Each class has a specific role and responsibility, and the cooperation between them will enable the entire system to operate.

2.2. Sequence Design

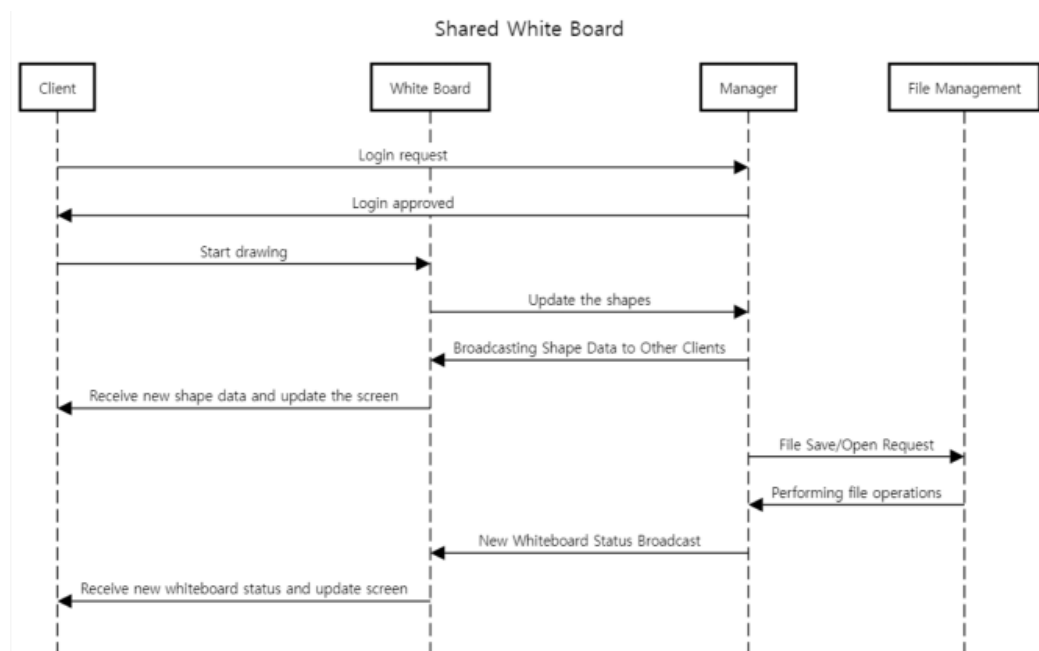


Figure 3. Sequence Diagram

① User login process:

- The client sends a connection request to the server.
- The manager approves or rejects the connection request.
- If the connection is approved, the client imports the current status of the whiteboard.
- Clients update their screens and maintain synchronization with their managers.

② Drawing process:

- The user draws a picture through the client interface.
- The client sends information about the figures drawn to the server.
- The server broadcasts the configuration information to all clients.
- All clients update the received configuration information on their screens.

③ File management process:

- The manager creates a new whiteboard through the File menu, or saves/loads an existing whiteboard.
- When the file management task is performed, the server synchronizes the current status of the whiteboard to all clients.
- All clients update their screens with the new whiteboard status.

3. Result

3.1. GUI

```
C:\Users\SOYEON PARK\Desktop\Assignment2\WhiteBoard>java -jar CreateWhiteBoard.jar 192.168.1.108 3000 ManagerA
WhiteBoard is ready to start on port 3000
```

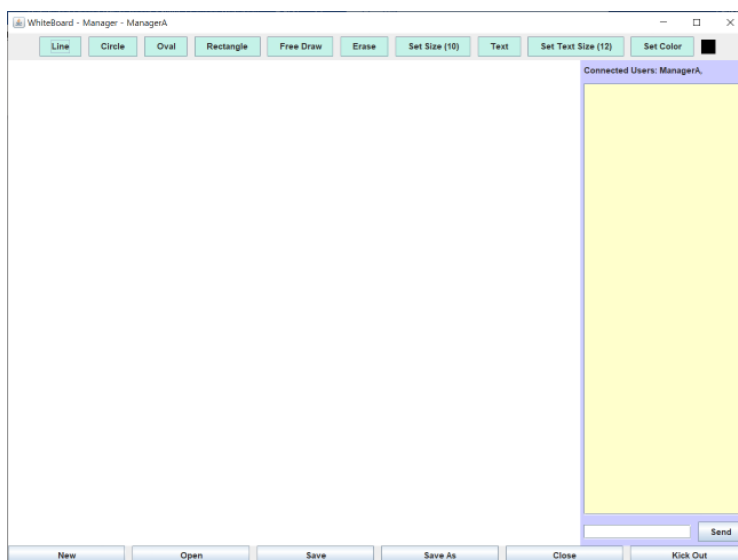


Figure 4. Manger's GUI

```
C:\Users\SOYEON PARK\Desktop\Assignment2\WhiteBoard>java -jar JoinWhiteBoard.jar 192.168.1.108 3000 UserB
User UserB joined the whiteboard.
```



Figure 5. Client's GUI

The above figures show the user interface of the shared whiteboard application. The interface is organized around a large whiteboard area where users can draw and input text.

The toolbar at the top includes buttons to draw various shapes. The user can draw lines, circles, oval, and square, as well as free drawing and erase functions. There are also buttons to set the size of lines and text, enter text, and select colors. The right panel displays a list of currently connected users. The "Connected Users" area displays the names of the currently connected users, which users can view in real time. At the bottom, there is a chat input field and a Send button, which allows users to send and receive messages to and from each other. The files menu at the bottom includes buttons to create, open, save, save as, close, and kick out a user. This menu is only visible from the manager's GUI and is used when the manager manages files on the whiteboard or expels certain users from the system.

Overall, this user interface is designed to allow users to intuitively access and take advantage of a variety of features.

3.2. Basic Functions

The system provides several tools for users to **draw different shapes**. Users can draw the desired shapes on a whiteboard by selecting **the line, circle, oval, and rectangle** tools from the toolbar. When drawing a figure, you can drag the mouse to adjust it to any size and position, making it intuitive and easy to use. In addition, a text tool allows users to **enter text** on a whiteboard. The entered text will appear in the selected location, and the sizes and colors can be adjusted to suit the user's needs. This feature allows users to add explanations and notes as well as drawings, making collaboration and communication smoother.

The **color selection** tool allows users to change the colors of shapes or text. A color palette using JColorChooser was provided to enable different color representations.

The system **synchronizes** the whiteboard status of all users in **real time**. When one user draws a picture or enters text, the content is immediately reflected to all others, so that all users can maintain the same whiteboard status. This synchronization improves collaboration efficiency and allows users to always view up-to-date whiteboards.

3.3. Additional Function

In addition to the basics, the system includes additional features to enhance collaboration and management. The **chat function** allows users to send and receive text messages in real time. Messages entered in the chat window are immediately sent to all users and displayed on the screen, allowing users to communicate smoothly while drawing.

The **Save and Open File** feature lets users save their current whiteboard status as a file or import an existing saved whiteboard file. You can do this by using the Manage Files option in the bottom menu, which saves all currently drawn figures and text when you save a file. This feature improves convenience by allowing you to save jobs in the middle and work on them again later.

In addition, the manager has the right to *kick out* certain users from the system. When a manager expels a user, the user is automatically logged out of the system, and the whiteboard is no longer accessible. This feature plays a critical role in managing the system and maintaining a smooth collaboration environment.

4. Conclusion

Through this project, I was able to successfully implement a multi-user shared whiteboard system. I was able to learn a lot from understanding and applying the basic concepts of remote method calls and network communication using Remote Method Invocation (RMI). In particular, I was able to feel the importance of synchronization and user management in building a system where multiple users connect at the same time and interact in real time.

As a result of the project, users can draw various shapes and input text through an intuitive GUI, and utilize various functions such as color selection. Real-time synchronization allows all users to maintain the same whiteboard state and smooth communication through chat. In addition, the file management function allows you to save and retrieve your work, greatly enhancing user convenience. Managers have the right to expel certain users, which allows them to effectively manage their systems.

However, there are also some limitations as the project progresses. For example, network delays can cause synchronization problems when there are many users, and the current system works smoothly only within the local network. To solve these problems, optimization tasks to minimize network delays and remote access capabilities through the Internet are needed.

As a future research plan, I am considering adding a function to **resize the whiteboard**. Currently, the whiteboard has a fixed size, but it can be adjusted to any size that the user wants to increase flexibility. In addition, I am also considering adding a **private chat** function by improving the chat function between users. This will allow users to send and receive messages individually in addition to group chat, allowing more detailed communication.

In the future, I plan to add these features to improve the user experience and further increase the utilization of the system. This project was a great help in enhancing the understanding of building distributed systems using RMI, and I was able to gain overall knowledge about distributed systems through the process of designing and implementing distributed applications that actually work. In the future, I would like to contribute to developing more efficient and scalable systems by developing and applying these technologies.